

컴퓨터구조 Computer Architecture

2장 - 데이터의 표현 김현석

Number System; 수체계; 진법(進法)

python 진법 변환기

```
number=input("Your Number: ")
base=int(input("Your Base: "))

try:
    if base==16 or base==10 or base==8 or base==2:
        dap=int(number, base)
        print("To Dec: ",dap)
        print("To Hex: ",hex(dap))
        print("To Oct: ",oct(dap))
        print("To Bin: ",bin(dap))
    else:
        print("not supported")
except ValueError:
    print("value not supported")
```

결과

```
Your Number: 23
Your Base: 8
To Dec: 19
To Hex: 0x13
To Oct: 0o23
To Bin: 0b10011
```

어떻게 읽어야 할까요?
'이십삼' or '이삼'

01 진법과 진법 변환

1 디지털 정보의 단위

- 1nibble = 4bit
- 1byte = 8bit
- 1byte = 1문자(character)
- 영어는 1byte로 1 문자 표현, 한글은 2byte가 필요
- 1워드 : 특정 CPU에서 취급하는 명령어나 데이터의 길이에 해당하는 비트 수
- 워드 길이는 8·16·32·64비트 등 8의 배수가 가능하다.

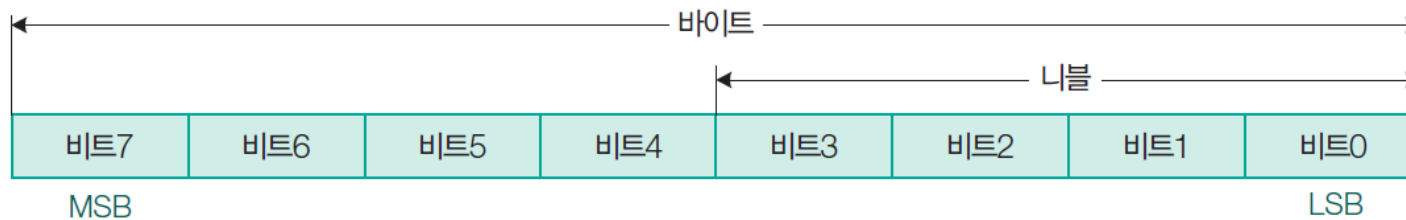


그림 2-1 비트, 니블, 바이트의 관계

MSB Most Significant Bit: 최상위 비트

LSB Least Significant Bit: 최하위 비트

01 진법과 진법 변환

❖ SI 단위와 IEC 단위 비교

SI(10진수 단위)			IEC(2진수 단위)			
값	기호	이름	값	기호	이름	10진수 변환 크기
$(10^3)^1 = 10^3$	k, K	kilo-	$(2^{10})^1 = 2^{10} \cong 10^{3.01}$	Ki	kibi-	1,024
$(10^3)^2 = 10^6$	M	mega-	$(2^{10})^2 = 2^{20} \cong 10^{6.02}$	Mi	mebi-	1,048,576
$(10^3)^3 = 10^9$	G	giga-	$(2^{10})^3 = 2^{30} \cong 10^{9.03}$	Gi	gibi-	1,073,741,824
$(10^3)^4 = 10^{12}$	T	tera-	$(2^{10})^4 = 2^{40} \cong 10^{12.04}$	Ti	tebi-	1,099,511,627,776
$(10^3)^5 = 10^{15}$	P	peta-	$(2^{10})^5 = 2^{50} \cong 10^{15.05}$	Pi	pebi-	1,125,899,906,842,624
$(10^3)^6 = 10^{18}$	E	exa-	$(2^{10})^6 = 2^{60} \cong 10^{18.06}$	Ei	exbi-	1,152,921,504,606,846,976
$(10^3)^7 = 10^{21}$	Z	zetta-	$(2^{10})^7 = 2^{70} \cong 10^{21.07}$	Zi	zebi-	1,180,591,620,717,411,303,424
$(10^3)^8 = 10^{24}$	Y	yotta-	$(2^{10})^8 = 2^{80} \cong 10^{24.08}$	Yi	yobi-	1,208,925,819,614,629,174,706,176

kibi-: kilobinary, mebi-: megabinary, gibi-: gigabinary,
tebi-: terabinary, pebi-: petabinary, exbi-: exabinary,
zebi-: zettabinary, yobi-: yottabinary

01 진법과 진법 변환

2 진법

❖ 10진법

- 10진수: 기수가 10인 수
- 0, 1, 2, 3, 4, 5, 6, 7, 8, 9의 10개 수로 표현

$$\begin{aligned} 9345.35 &= 9 \times 1000 + 3 \times 100 + 4 \times 10 + 5 \times 1 + 3 \times 0.1 + 5 \times 0.01 \\ &= 9 \times 10^3 + 3 \times 10^2 + 4 \times 10^1 + 5 \times 10^0 + 3 \times 10^{-1} + 5 \times 10^{-2} \end{aligned}$$

❖ 2진법

- 기수가 2인 수
- 0, 1 두 개의 수로 표현

$$\begin{aligned} 1010.1011_{(2)} &= 1 \times 1000_{(2)} + 0 \times 100_{(2)} + 1 \times 10_{(2)} + 0 \times 1_{(2)} \\ &\quad + 1 \times 0.1_{(2)} + 0 \times 0.01_{(2)} + 1 \times 0.001_{(2)} + 1 \times 0.0001_{(2)} \\ &= 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} + 1 \times 2^{-4} \\ &= 10.6875 \text{ (10진법)} \end{aligned}$$

01 진법과 진법 변환

❖ 8진법

- 0~7까지 8개의 수로 표현

$$\begin{aligned} 607.36_{(8)} &= 6 \times 100_{(8)} + 0 \times 10_{(8)} + 7 \times 1_{(8)} + 3 \times 0.1_{(8)} + 6 \times 0.01_{(8)} \\ &= 6 \times 8^2 + 0 \times 8^1 + 7 \times 8^0 + 3 \times 8^{-1} + 6 \times 8^{-2} \end{aligned}$$

❖ 16진법

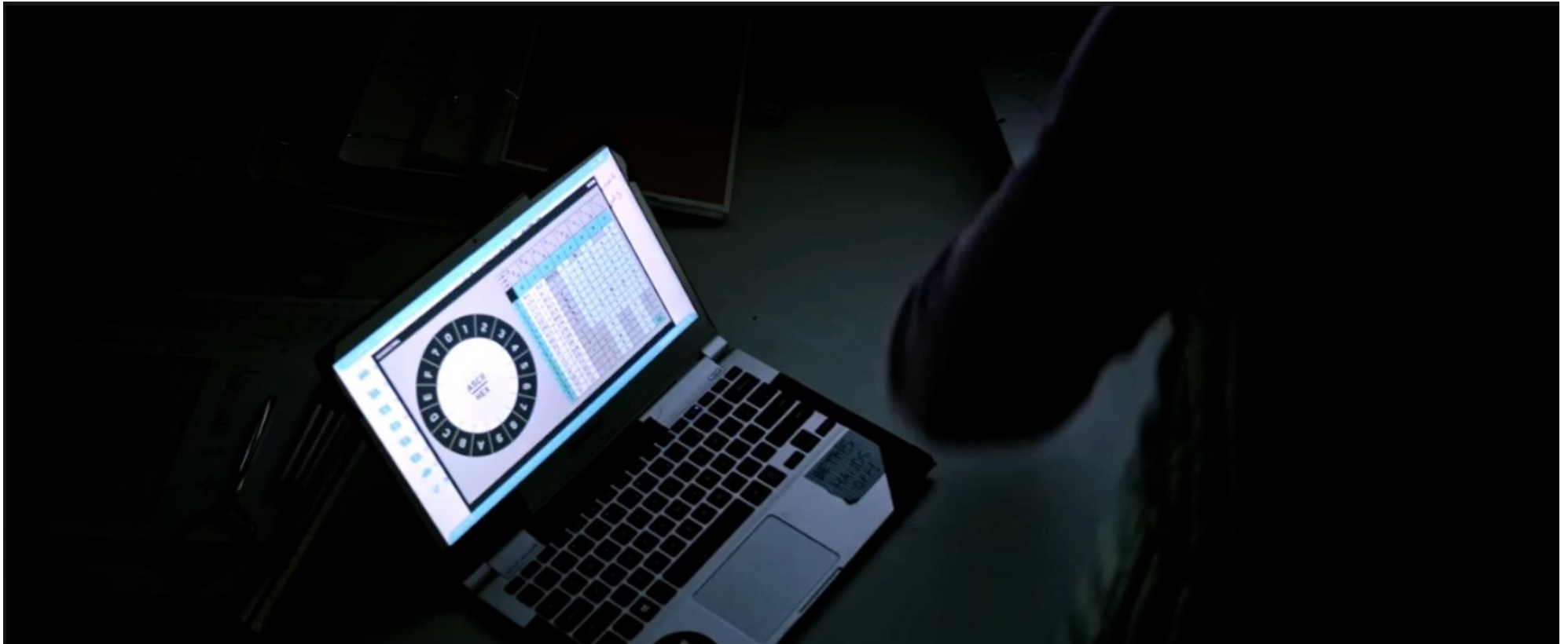
- 0~9, A~F까지 16개의 기호로 표현

16진법: 한자리 숫자로 더 많은 경우를 표현할 수 있음

$$\begin{aligned} 6C7.3A_{(16)} &= 6 \times 100_{(16)} + C \times 10_{(16)} + 7 \times 1_{(16)} + 3 \times 0.1_{(16)} + A \times 0.01_{(16)} \\ &= 6 \times 16^2 + C \times 16^1 + 7 \times 16^0 + 3 \times 16^{-1} + A \times 16^{-2} \end{aligned}$$

- 8진수보다는 16진수를 사용하는 경우가 더 많은데 실제로 컴퓨터 구조나 어셈블리어에서는 16진수를 많이 쓴다. 자릿수를 더 짧게 표현할 수 있기 때문이다.

16진수; Hex Code; 영화 '마션'



01 진법과 진법 변환

표 2-2 2진수에 해당하는 8진수, 16진수, 10진수 표현

2진수	8진수	10진수	2진수	16진수	10진수	2진수	16진수	10진수
000	0	0	0000	0	0	1000	8	8
001	1	1	0001	1	1	1001	9	9
010	2	2	0010	2	2	1010	A	10
011	3	3	0011	3	3	1011	B	11
100	4	4	0100	4	4	1100	C	12
101	5	5	0101	5	5	1101	D	13
110	6	6	0110	6	6	1110	E	14
111	7	7	0111	7	7	1111	F	15

ASCII Code(Hex)

ASCII CODE TABLE

10	HEX	문자	10	HEX	문자	10	HEX	문자	10	HEX	문자	10	HEX	문자	10	HEX	문자
0	0x00	NULL	22	0x16	STN	44	0x2C	,	66	0x42	B	88	0x58	X	110	0x6E	n
1	0x01	SOH	23	0x17	ETB	45	0x2D	-	67	0x43	C	89	0x59	Y	111	0x6F	o
2	0x02	STX	24	0x18	CAN	46	0x2E	.	68	0x44	D	90	0x5A	Z	112	0x70	p
3	0x03	ETX	25	0x19	EM	47	0x2F	/	69	0x45	E	91	0x5B	[	113	0x71	q
4	0x04	EOT	26	0x1A	SUB	48	0x30	0	70	0x46	F	92	0x5C	\	114	0x72	r
5	0x05	ENQ	27	0x1B	ESC	49	0x31	1	71	0x47	G	93	0x5D	]	115	0x73	s
6	0x06	ACK	28	0x1C	FS	50	0x32	2	72	0x48	H	94	0x5E	^	116	0x74	t
7	0x07	BEL	29	0x1D	GS	51	0x33	3	73	0x49	I	95	0x5F	_	117	0x75	u
8	0x08	BS	30	0x1E	RS	52	0x34	4	74	0x4A	J	96	0x60	.	118	0x76	v
9	0x09	HT	31	0x1F	US	53	0x35	5	75	0x4B	K	97	0x61	a	119	0x77	w
10	0x0A	LF	32	0x20	SP	54	0x36	6	76	0x4C	L	98	0x62	b	120	0x78	x
11	0x0B	VT	33	0x21	!	55	0x37	7	77	0x4D	M	99	0x63	c	121	0x79	y
12	0x0C	FF	34	0x22	"	56	0x38	8	78	0x4E	N	100	0x64	d	1222	0x7A	z
13	0x0D	CR	35	0x23	#	57	0x39	9	79	0x4F	O	101	0x65	e	123	0x7B	{
14	0x0E	SO	36	0x24	\$	58	0x3A	:	80	0x50	P	102	0x66	f	124	0x7C	
15	0x0F	SI	37	0x25	%	59	0x3B	;	81	0x51	Q	103	0x67	g	125	0x7D	}
16	0x10	DEL	38	0x26	&	60	0x3C	<	82	0x52	R	104	0x68	h	126	0x7E	~
17	0x11	DC1	39	0x27	'	61	0x3D	=	83	0x53	S	105	0x69	i	127	0x7F	DEL
18	0x12	DC2	40	0x28	(	62	0x3E	>	84	0x54	T	106	0x6A	j			
19	0x13	DC3	41	0x29	)	63	0x3F	?	85	0x55	U	107	0x6B	k			
20	0x14	DC4	42	0x2A	*	64	0x40	@	86	0x56	V	108	0x6C	l			
21	0x15	NAK	43	0x2B	+	65	0x41	A	87	0x57	W	109	0x6D	m			

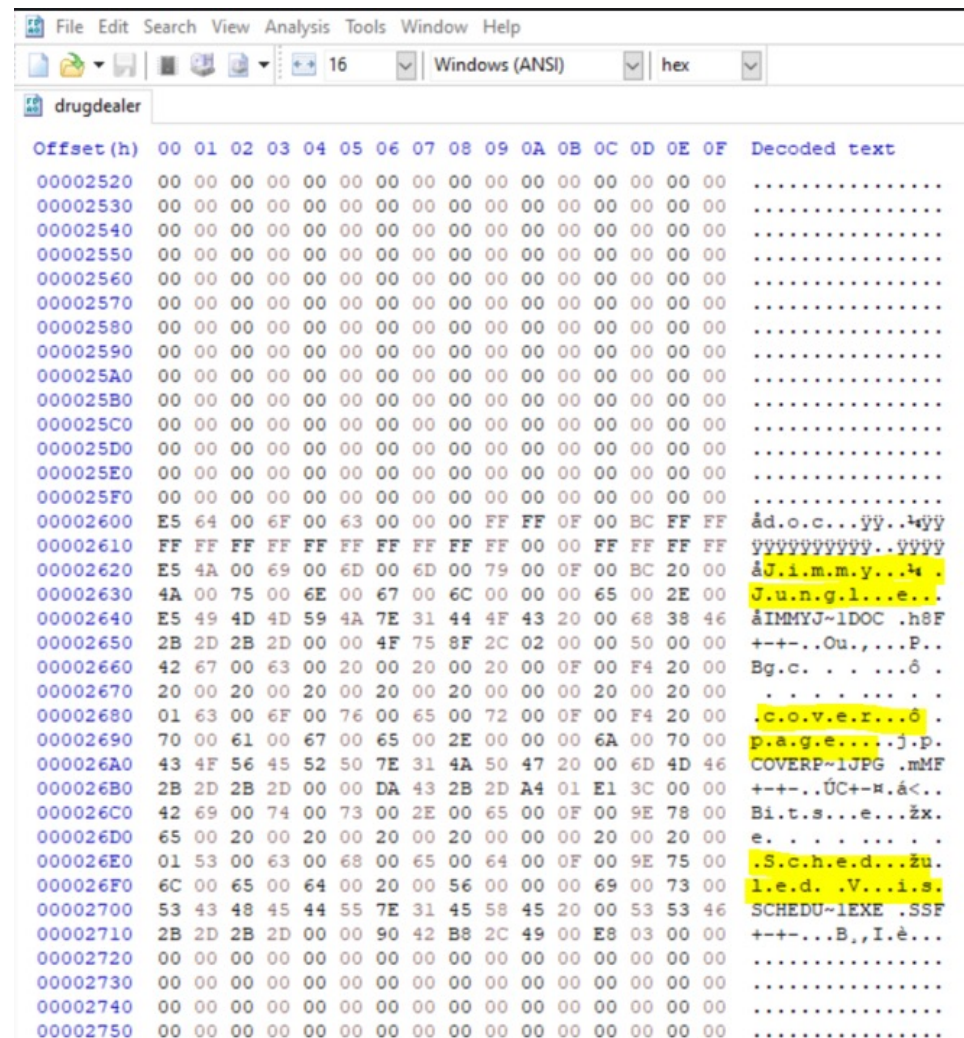
그림 2.2 ASCII 코드표

외우는 법:

0x31: 1 ~ 0x39: 9

0x41: A ~ 0x5A: Z

0x61: a ~ 0x7A: z



디지털 포렌식 한 예

01 진법과 진법 변환

3 진법 변환

□ 2진수, 8진수, 16진수를 10진수로 변환

① 2진수를 10진수로 변환한 예

$$\begin{aligned}101101.101_{(2)} &= 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} \\&= 32 + 0 + 8 + 4 + 0 + 1 + 0.5 + 0 + 0.125 \\&= 45.625_{(10)}\end{aligned}$$

② 8진수를 10진수로 변환한 예

$$\begin{aligned}364.35_{(8)} &= 3 \times 8^2 + 6 \times 8^1 + 4 \times 8^0 + 3 \times 8^{-1} + 5 \times 8^{-2} \\&= 3 \times 64 + 6 \times 8 + 4 \times 1 + 3 \times 0.125 + 5 \times 0.015625 \\&= 192 + 48 + 4 + 0.375 + 0.078125 \\&= 244.453125_{(10)}\end{aligned}$$

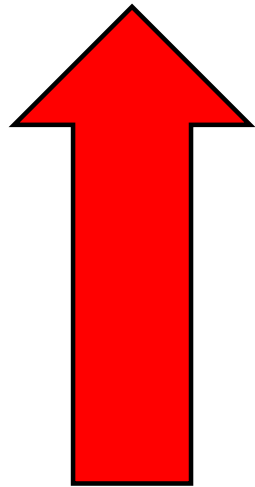
③ 16진수를 10진수로 변환한 예

$$\begin{aligned}A3.D2_{(16)} &= 10 \times 16^1 + 3 \times 16^0 + 13 \times 16^{-1} + 2 \times 16^{-2} \\&= 10 \times 16 + 3 \times 1 + 13 \times 0.0625 + 2 \times 0.00390625 \\&= 160 + 3 + 0.8125 + 0.0078125 \\&= 163.8203125_{(10)}\end{aligned}$$

01 진법과 진법 변환

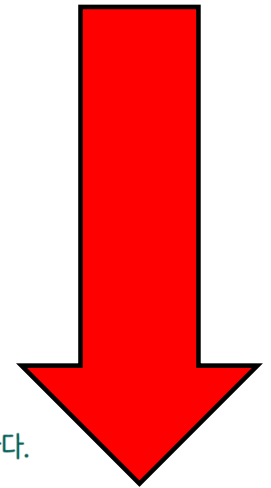
□ 10진수 -> 2진수 변환

- 정수부분과 소수부분으로 나누어 변환
- 정수부분은 2로 나누고, 소수부분은 2를 곱한다.
- 10진수 75.6875를 2진수로 변환한 예



2	75	나머지	→	2진수
2	37	...	1	→ 1
2	18	...	1	→ 11
2	9	...	0	→ 011
2	4	...	1	→ 1011
2	2	...	0	→ 01011
2	1	...	0	→ 001011
	0	...	1	→ 1001011
몫				

2진수	←	정수	소수	
			0.	6875
			×	2
				2
0.1	←	1.	3750	곱셈 결과 정수를 적는다.
		×	2	
			2	
0.10	←	0.	7500	
		×	2	
			2	
0.101	←	1.	5000	
		×	2	
			2	
0.1011	←	1.	0	소수 부분이 0이 될 때까지 계산한다.

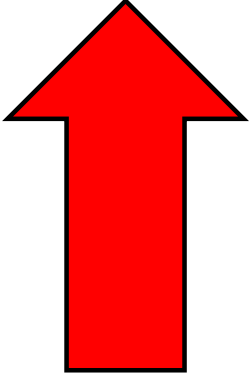


$$75.6875_{(10)} = 1001011.1011_{(2)}$$

01 진법과 진법 변환

□ 10진수 -> 8진수 변환

- 10진수 75.6875를 8진수로 변환
- 정수부분은 8로 나누고, 소수부분은 8을 곱한다.

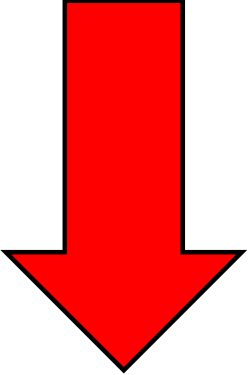


8	75	나머지	→ 8진수
8	9 ...	3	→ 3
8	1 ...	1	→ 13
	0 ...	1	→ 113
	몫		

8진수	← 정수	소수
	0.	6875
	×	8
0.5	← 5.	5000
	×	8
0.54	← 4.	0

곱셈 결과 정수를 적는다.

소수 부분이 0이 될 때까지 계산한다.

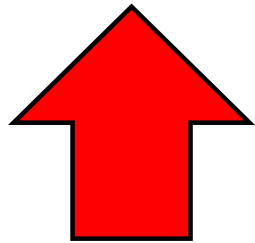


$$75.6875_{(10)} = 113.54_{(8)}$$

01 진법과 진법 변환

□ 10진수 -> 16진수 변환

- 10진수 75.6875를 16진수로 변환
- 정수부분은 16으로 나누고, 소수부분은 16을 곱한다.

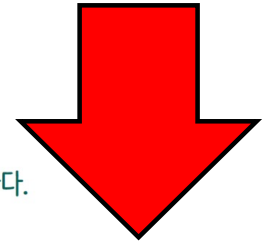


16		75		나머지	→	16진수
16		4	...	11	→	B
		0	...	4	→	4B
		몫				

16진수	←	정수	소수
		0.	6875
		×	16
0.B	←	11.	0000

곱셈 결과 정수를 적는다.

소수 부분이 0이 될 때까지 계산한다.



$$75.6875_{(10)} = 4B.B_{(16)}$$

01 진법과 진법 변환

□ 2진수 - 8진수 - 10진수 - 16진수 상호 변환

먼저 2진수로 변환해서 다른
진법으로 변환하는 방법이 편함

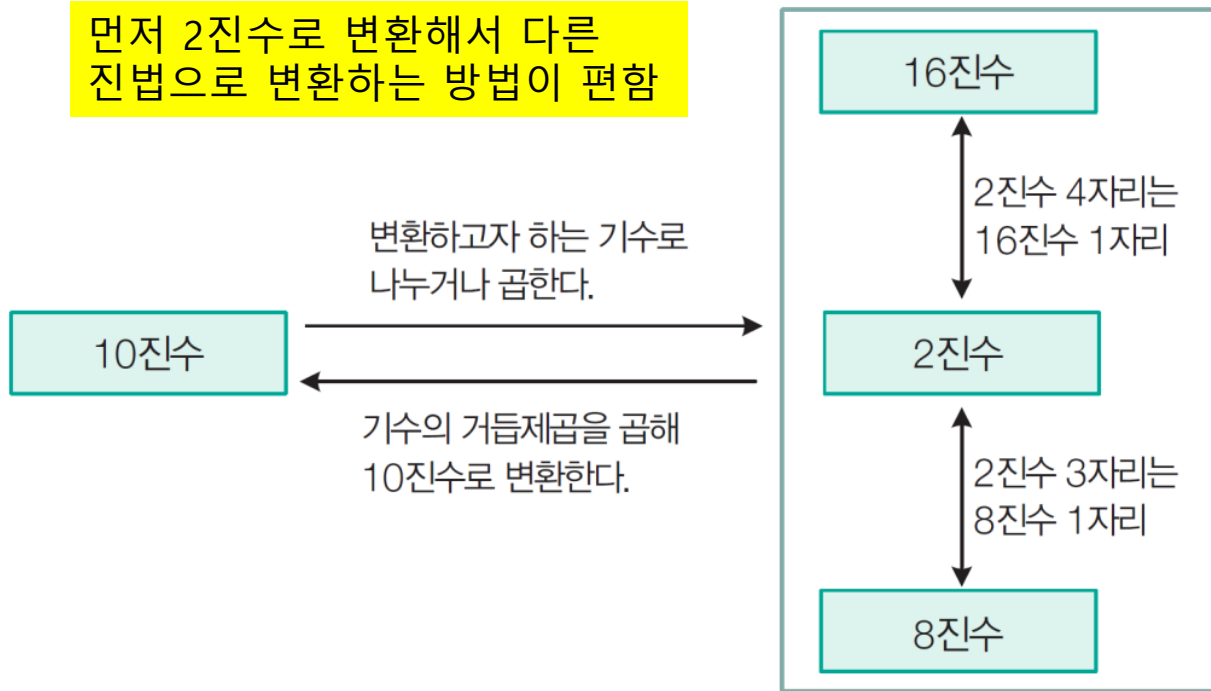


그림 2-2 2진수, 8진수, 10진수, 16진수 상호 변환 개념도

10진수	2진수	8진수	16진수
0	0000	00	0
1	0001	01	1
2	0010	02	2
3	0011	03	3
4	0100	04	4
5	0101	05	5
6	0110	06	6
7	0111	07	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

01 진법과 진법 변환

- 10진수를 8진수, 16진수로 변환한 예

$$\begin{aligned} 75.6875 &= 1001011.1011_{(2)} \\ &= \textcolor{teal}{001} \ 001 \ 011.101 \ \textcolor{teal}{100}_{(2)} \\ &= \quad 1 \quad 1 \quad 3. \ 5 \quad 4_{(8)} \end{aligned}$$

$$\begin{aligned} 75.6875 &= 1001011.1011_{(2)} \\ &= \textcolor{teal}{0}100 \ 1011.1011_{(2)} \\ &= \quad 4 \quad B. \ B_{(16)} \end{aligned}$$

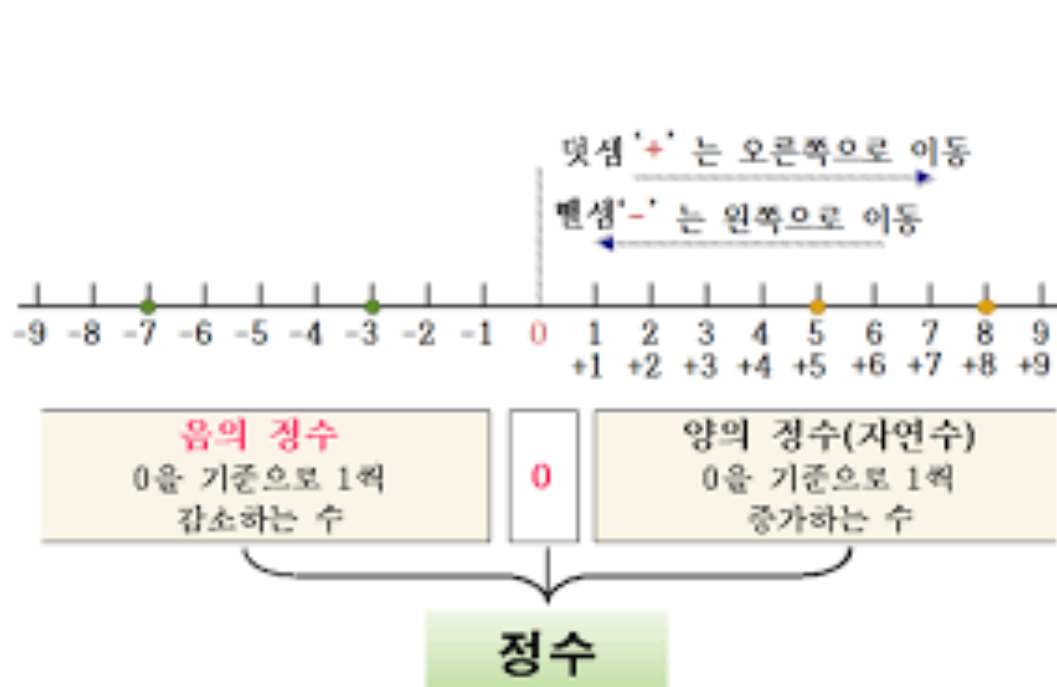
- 8진수와 16진수를 2진수로 변환한 예

$$\begin{aligned} &\quad 3 \quad 6 \quad 7. \quad 7 \quad 5_{(8)} \\ &= 011 \ 110 \ 111. \ 111 \ 101_{(2)} \end{aligned}$$

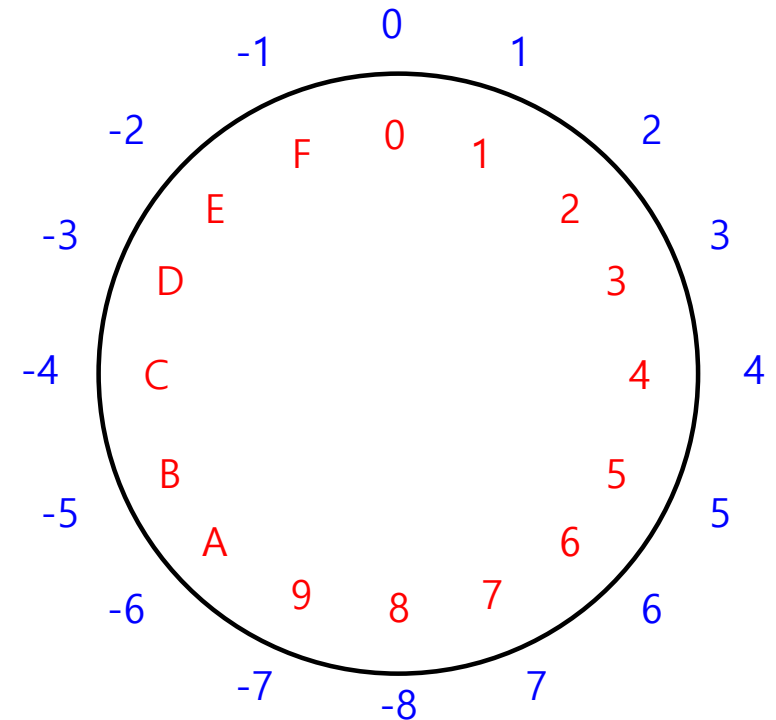
$$\begin{aligned} &\quad 9 \quad A \quad 3. \quad 5 \quad 0 \quad F \quad 3_{(16)} \\ &= 1001 \ 1010 \ 0011. \ 0101 \ 0000 \ 1111 \ 0011_{(2)} \end{aligned}$$

보수; 보충하는 수; Complement Number System
컴퓨터에서 음수를 표현하기 위해서 보수라는 개념을 사용
ALU에 뺄셈 회로가 없었음. 보수를 더하는 것이 뺄셈과 같은 효과

수학에서의 수체계와 컴퓨터에서의 수체계



수학에서는 무한대의
수가 존재



컴퓨터에서는
유한개의 수가 존재
(순환 버퍼의 개념)

02 정수 표현

1 보수의 개념과 음수

❖ 최상위비트(MSB)를 부호비트로 사용

- 양수(+): 0 음수(-): 1

❖ 2진수 음수를 표시하는 방법

- 부호와 절댓값(sign-magnitude)
- 1의 보수(1's complement)
- 2의 보수(2's complement)

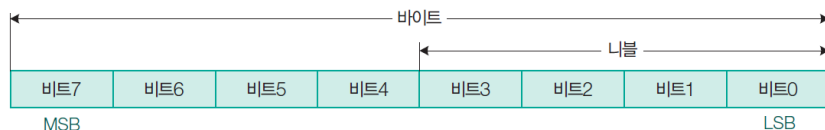


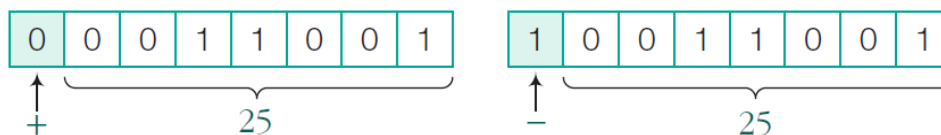
그림 2-1 비트, 니블, 바이트의 관계

MSB: Most Significant Bit: 최상위 비트
LSB: Least Significant Bit: 최하위 비트

컴퓨터에서는
음수를 나타내기 위해
2의 보수 방식을 사용함

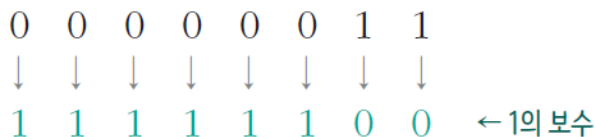
❖ 부호와 절댓값

- 부호비트만 양수와 음수를 나타내고 나머지 비트들은 같다.



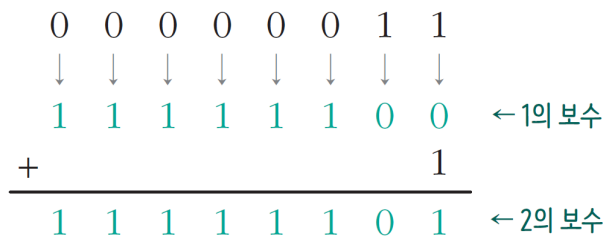
❖ 1의 보수 방식

- 0 → 1, 1 → 0으로 변환



❖ 2의 보수 방식

- 1의 보수 + 1 = 2의 보수



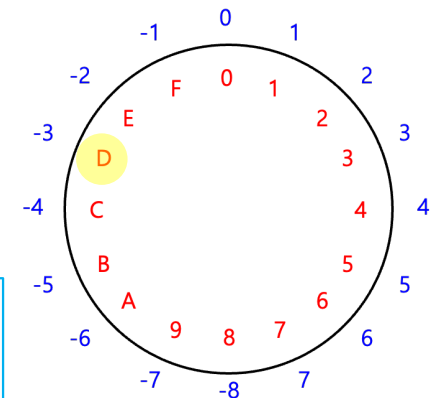
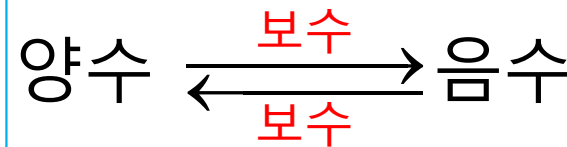
02 정수 표현

- ❖ r 진수에는 r 의 보수와 $r-1$ 의 보수가 있다.
 - 10진수에는 9의 보수와 10의 보수가 있고
8진수에는 7의 보수와 8의 보수가 있으며
2진수에는 1의 보수와 2의 보수가 있다.
 - 567의 9의 보수 : $999-567=432$
 - 567의 10의 보수 : $1000-567=433$ ($=432+1$; 567의 9의 보수에 +1)
 - 0011의 1의 보수 : $1111-0011=1100$
 - 0011의 2의 보수 : $10000-0011=1101$ ($=1100+1$; 0011의 1의 보수에 +1)

컴퓨터 유한개의 수 체계에서는

- ❖ 양수를 보수로 바꾸면 음수: 0011(+3) → 1101(-3)
- ❖ 음수를 보수로 바꾸면 양수: 1101(-3) → 0011(+3)

D



컴퓨터에서는
유한개의 수가 존재
(순환 버퍼의 개념)

- ❖ 2진수와 그 수의 1의 보수와 합은 모든 bit가 1이 된다. ($0011 + 1100 = 1111$)
- ❖ 2진수와 그 수의 2의 보수와 합은 모든 bit가 0이 된다. ($0011 + 1101 = 10000$)
(자릿수를 벗어나는 비트는 제외)

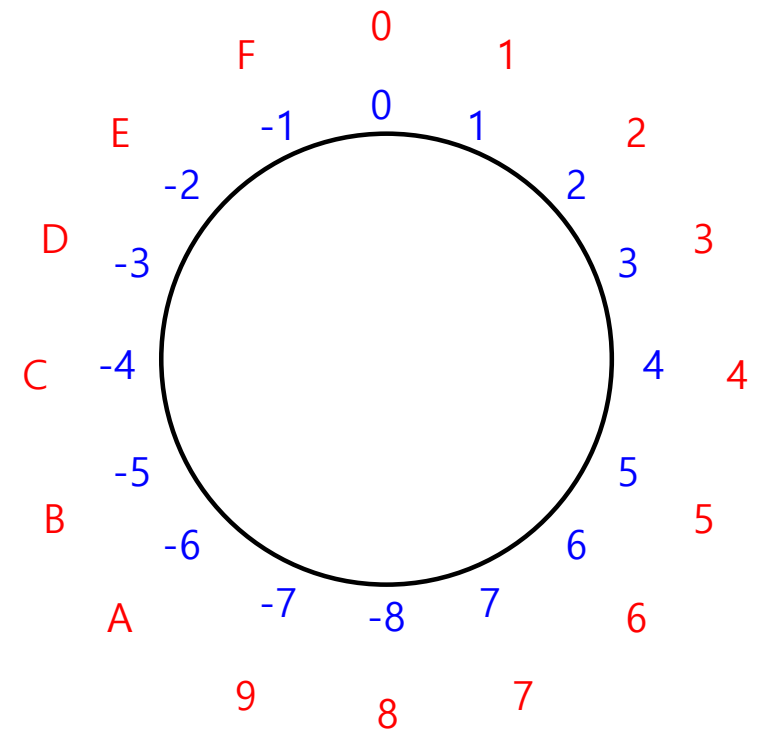
뺄셈 == 보수를 이용한 덧셈

- 10진수 한자리 숫자의 경우 숫자 3에 10의 보수는 7
- $5 - 3 = 2$
- $5 + 7 = 12$ (12는 두자리 숫자)
- 이때, 올림(carry) 발생하여 다음 자리로 숫자가 넘어 가는 것임
- 16진수 한자리 숫자의 경우 숫자 9에 16의 보수는 7
- $A - 9 = 1$
- $A + 7 = 1$
- 2진수 네자리 숫자의 경우 숫자 1001에 2의 보수는 0111
- $1100 - 1001 = 0011$
- $1100 + 0111 = 0011$

02 정수 표현

❖ 수의 표현 방법에 따른 10진수 대응 값

4비트 2진수	부호 없는 수	부호와 절댓값	1의 보수	2의 보수
0000	0	0	0	0
0001	1	1	1	1
0010	2	2	2	2
0011	3	3	3	3
0100	4	4	4	4
0101	5	5	5	5
0110	6	6	6	6
0111	7	7	7	7
1000	8	-0	-7	-8
1001	9	-1	-6	-7
1010	10	-2	-5	-6
1011	11	-3	-4	-5
1100	12	-4	-3	-4
1101	13	-5	-2	-3
1110	14	-6	-1	-2
1111	15	-7	-0	-1



데이터형(Data Type) 나온 이유?

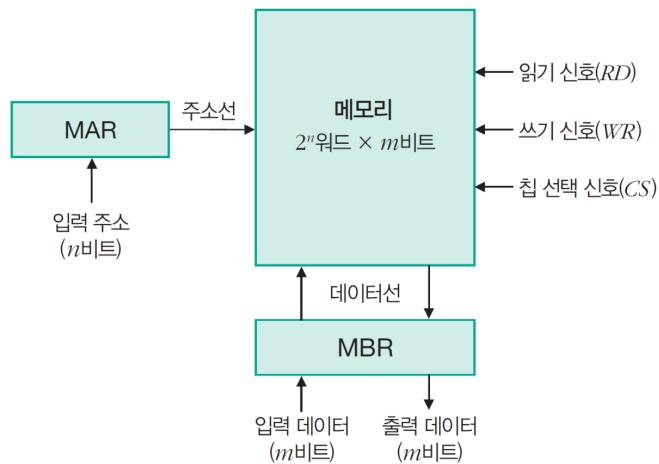


그림 6-2 주기억 장치의 동작 블록도

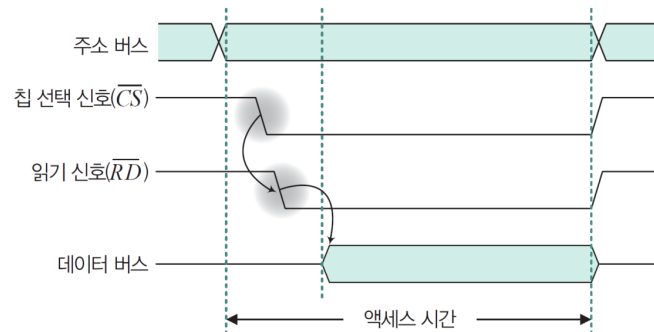


그림 6-3 메모리 읽기 동작의 타이밍도

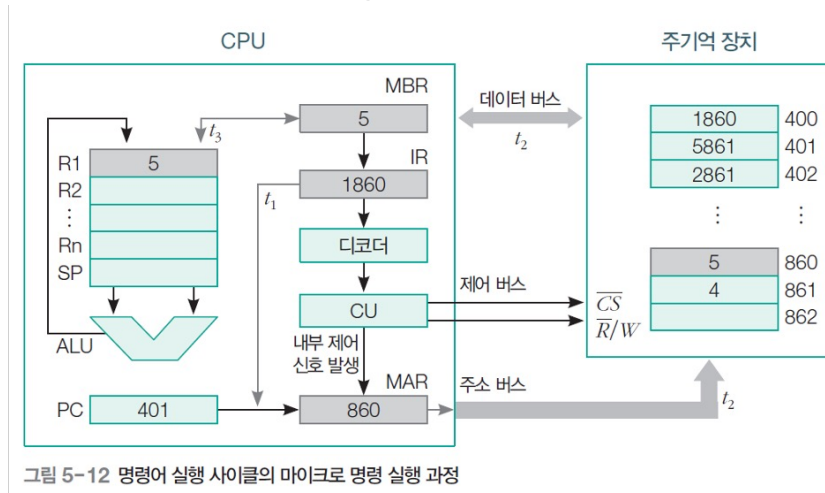
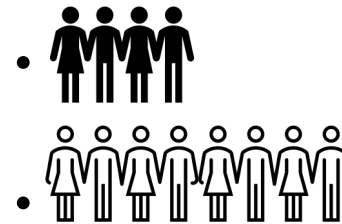
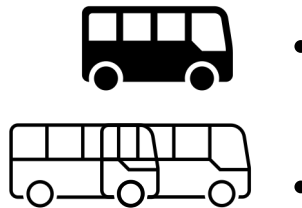


그림 5-12 명령어 실행 사이클의 마이크로 명령 실행 과정

- t_1 : $MAR \leftarrow (IR:operand)$; 명령 레지스터의 오퍼랜드(860)를 MAR로
 t_2 : $MBR \leftarrow M[MAR]$; 메모리에서 피연산자를 읽어 MBR로
 t_3 : $R1 \leftarrow (MBR)$; MBR에서 R1(누산기 역할)로

다음 중 인원수에 따른 가장 효율적인 버스 배차는?



컴퓨터 자원을 효율적으로 사용하기 위해 컴파일러가 적절한 명령으로 변환할 수 있도록 사용자로 하여금 데이터형을 먼저 정의하도록 함

02 정수 표현

❖ 2의 보수에 대한 10진수의 표현 범위

표 2-4 n 비트 2의 보수에 대한 10진수의 표현 범위

비트 수	2의 보수를 사용한 2진 정수의 표현 범위
n 비트	$-2^{n-1} \sim +2^{n-1}-1$
4비트	$-2^{4-1}(-8) \sim +2^{4-1}-1(+7)$
8비트	$-2^{8-1}(-128) \sim +2^{8-1}-1(+127)$
16비트	$-2^{16-1}(-32,768) \sim +2^{16-1}-1(+32,767)$
32비트	$-2^{32-1}(-2,147,483,648) \sim +2^{32-1}-1(+2,147,483,647)$
64비트	$-2^{64-1}(-9,223,372,036,854,775,808) \sim +2^{64-1}-1(+9,223,372,036,854,775,807)$

02 정수 표현

2 부호 확장

- 부호 확장이란 늘어난 비트 수 만큼 부호를 늘려주는 방법

표 2-5 2진수 표현 방법에 따른 부호 확장

2진수 표현 방식	부호 확장 방법	구분	8비트	16비트 확장
부호와 절댓값	부호만 MSB에 복사하고, 나머지는 0으로 채운다.	양수	00101010	00000000 00101010
		음수	10010111	10000000 00010111
1의 보수	늘어난 길이만큼 부호와 같은 값으로 모두 채운다.	양수	00101010	00000000 00101010
		음수	10010111	11111111 10010111
2의 보수	늘어난 길이만큼 부호와 같은 값으로 모두 채운다.	양수	00101010	00000000 00101010
		음수	10010111	11111111 10010111

02 정수 표현

3 2진 정수 연산

- 뺄셈의 원리를 보면, A-B 대신에 A+(B의 2의 보수)를 계산하면 된다.
- 뺄셈에서 2의 보수 방식을 사용하는 이유는 뺄셈을 가산기를 사용하여 수행할 수 있기 때문

① 자리올림수 → 0110000

$$\begin{array}{r} \text{양수} \quad 49 = 00110001 \\ + \text{양수} \quad +58 = + 00111010 \\ \hline \text{양수} \quad 107 = 0\ 01101011 \end{array}$$

② 자리올림수 → 1111110

$$\begin{array}{r} \text{양수} \quad 58 = 00111010 \\ - \text{양수} \quad -49 = - 00110001 \\ \hline \text{양수} \quad \quad \quad 00111010 \\ + \text{음수} \quad \quad \quad + 11001111 \quad \text{2의 보수} \\ \hline \text{양수} \quad \quad 9 = 1\ 00001001 \end{array}$$

③ 자리올림수 → 0000000

$$\begin{array}{r} \text{양수} \quad 49 = 00110001 \\ - \text{양수} \quad -58 = - 00111010 \\ \hline \text{양수} \quad \quad \quad 00110001 \\ + \text{음수} \quad \quad \quad + 11000110 \quad \text{2의 보수} \\ \hline \text{음수} \quad -9 = 0\ 11110111 \end{array}$$

④ 자리올림수 → 1001110

$$\begin{array}{r} - \text{양수} \quad -49 = - 00110001 \\ - \text{양수} \quad -58 = - 00111010 \\ \hline \text{음수} \quad \quad \quad 11001111 \\ + \text{음수} \quad \quad \quad + 11000110 \quad \text{2의 보수} \\ \hline \text{음수} \quad -107 = 1\ 10010101 \end{array}$$

⑤ 자리올림수 → 1000010

$$\begin{array}{r} \text{양수} \quad 98 = 01100010 \\ + \text{양수} \quad +74 = + 01001010 \\ \hline \text{음수} \quad -84 = 0\ 10101100 \end{array}$$

+172 서로
다름

⑥ 자리올림수 → 0111110

$$\begin{array}{r} - \text{양수} \quad -98 = - 01100010 \\ - \text{양수} \quad -74 = - 01001010 \\ \hline \text{음수} \quad \quad \quad 10011110 \\ + \text{음수} \quad \quad \quad + 10110110 \quad \text{2의 보수} \\ \hline \text{양수} \quad +84 = 1\ 01010100 \end{array}$$

서로
다름 -172

signed byte(8비트)
표현할 수 있는 수의 범위는
-128 ~ 127

overflow

32비트	$-2^{32-1}(-2,147,483,648) \sim +2^{32-1}-1(+2,147,483,647)$
------	--

**signed int의 범위를 벗어나면
(개발자에게는 등골이 오싹한) overflow;underflow 발생**

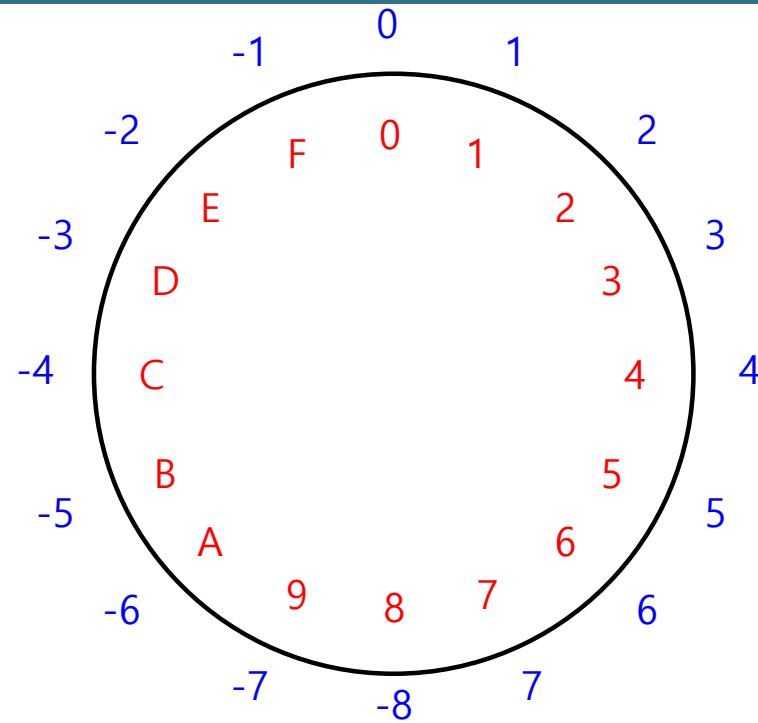


02 정수 표현

❖ 수의 표현 방법에 따른 10진수 대응 값

4비트 2진수	부호 없는 수	2의 보수
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	-8
1001	9	-7
1010	10	-6
1011	11	-5
1100	12	-4
1101	13	-3
1110	14	-2
1111	15	-1

Underflow ↑ ↓ Overflow



Underflow; 언더플로우

최소값보다 낮아지지 못하고 최대값으로 가는 경우

Overflow; 오버플로우

최대값보다 높아지지 못하고 최소값으로 가는 경우

overflow, underflow

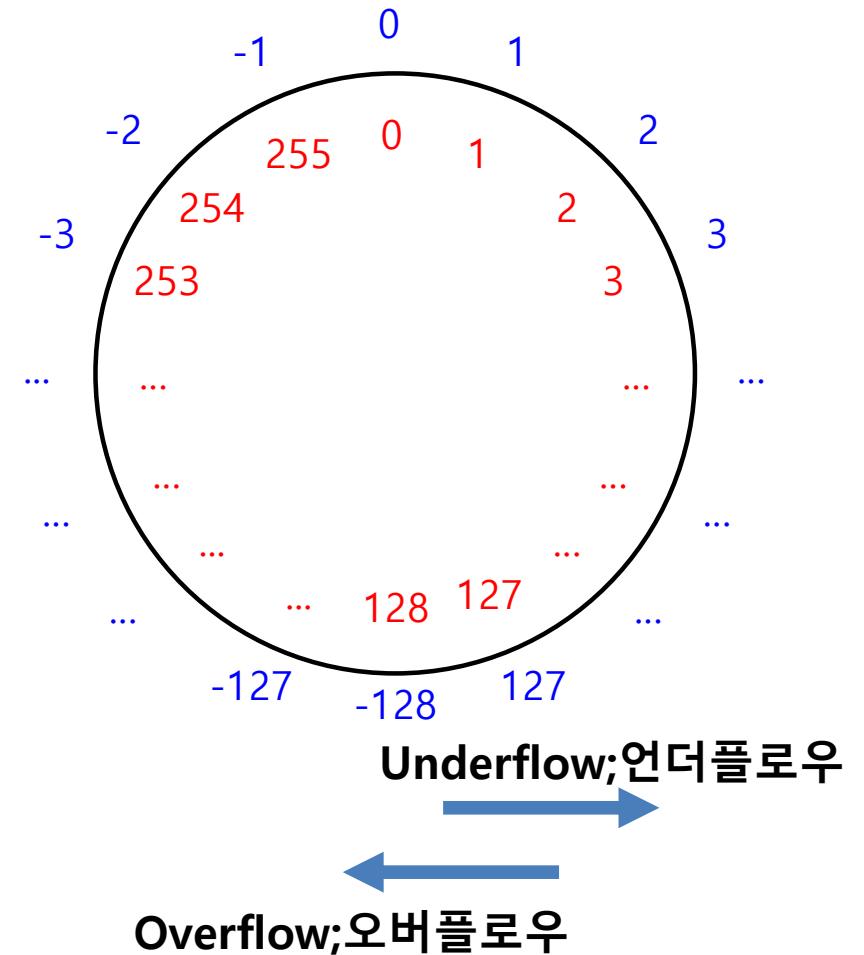
```
unsigned char ubVar=0;
signed char bVar=0;

for(int i=0; i<257; i++)
{
    printf("%d ", ubVar);
    ubVar++;
}
printf("\n");

for(int i=0; i<257; i++)
{
    printf("%d ", bVar);
    bVar++;
}
printf("\n");
```

```
ubVar = 255;
for(int i=0; i<257; i++)
{
    printf("%d ", ubVar);
    ubVar--;
}
printf("\n");

bVar =255;
for(int i=0; i<257; i++)
{
    printf("%d ", bVar);
    bVar--;
}
```



overflow, underflow

```
unsigned char ubVar=0;
signed char bVar=0;

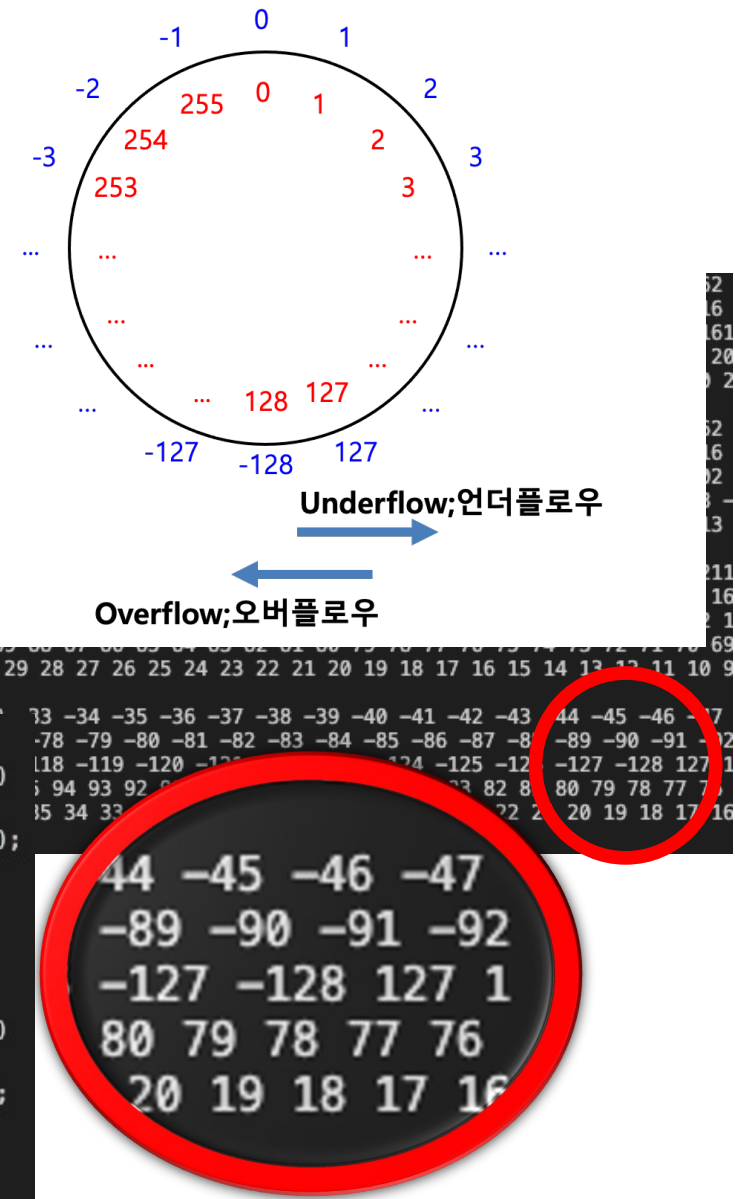
for(int i=0; i<257; i++)
{
    printf("%d ", ubVar);
    ubVar++;
}
printf("\n");

for(int i=0; i<257; i++)
{
    printf("%d ", bVar);
    bVar++;
}
printf("\n");
```

Hyunseoks-MacBook-Pro:코드작업 hertzkim\$

```
ubVar = 255;
for(int i=0; i<257; i++)
{
    printf("%d ", ubVar);
    ubVar--;
}
printf("\n");

bVar = 255;
for(int i=0; i<257; i++)
{
    printf("%d ", bVar);
    bVar--;
}
```

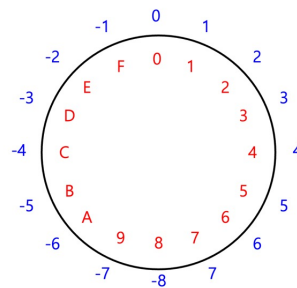


실수; Floating Point Number; 소수점이 고정되지 않고 움직이는 부동(浮動부 動움질일 동)

수학에서의 수체계와 컴퓨터에서의 수체계



수학에서는 무한대의 수가 존재



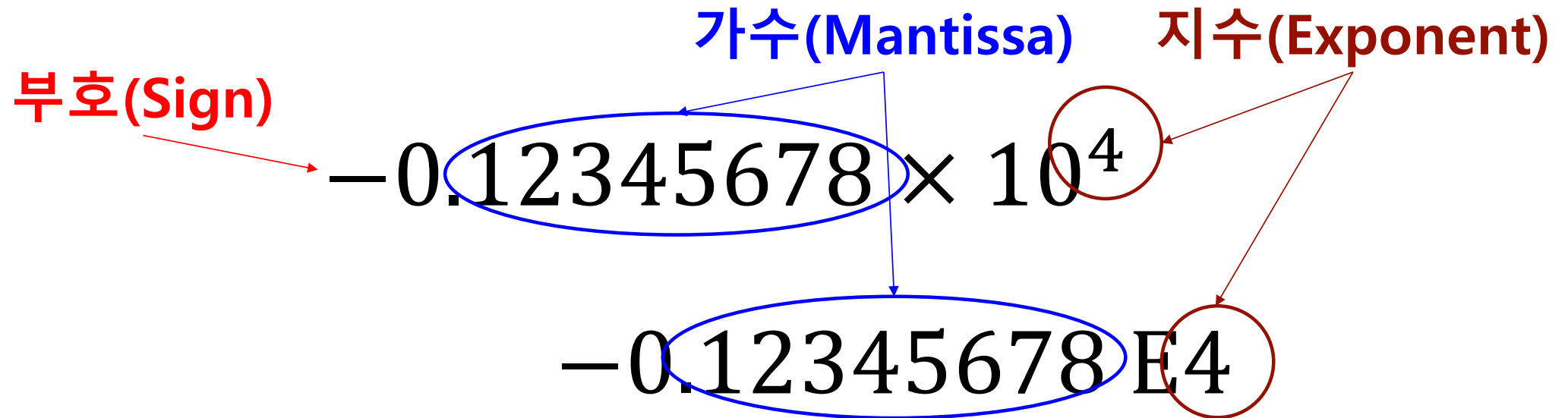
컴퓨터에서는 유한개의 수가 존재 (순환 버퍼의 개념)

고정소수점: 표현할 수 있는 실수의 범위가 고정
부동소수점: 표현할 수 있는 실수의 범위가 부동

그러면, 어떻게 가능한가요? → 소수점을 이동시킴
적은 비트로 훨씬 더 넓은 범위의 실수 표현이 가능해짐

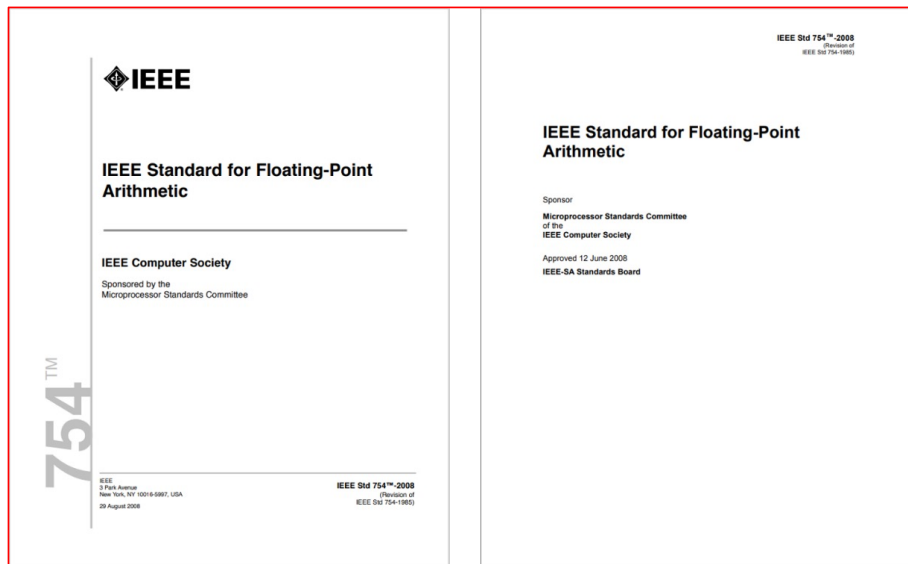
부호, 지수, 가수

-1234.5678



03 실수 표현

- 컴퓨터의 **부동소수점수**는 **IEEE 754표준**을 따른다.
- **부호**(sign), **지수**(exponent), **가수**(mantissa; fraction)의 세 영역으로 표시
- 부호(S)가 0일 때는 양수를 나타내고, 1일 때는 음수를 나타낸다.
- 단 정 도 (single precision) 부 동 소 수 점 수 와 배 정 도 (double precision) 부동소수점수의 두 가지 표현 방법이 있다.



<https://irem.univ-reunion.fr/IMG/pdf/ieee-754-2008.pdf>

단정도(Single Precision), 배정도(Double Precision)

바이어스: 지수의 음/양을 구분하기 위해 바이어스를 이용: 2^{10} , 2^{-10}
 $127+10$, $127-10$

구분	IEEE 754 표준 부동 소수점 수의 비트 할당	바이어스
단정도 부동 소수점 수		127
배정도 부동 소수점 수		1023

그림 2-5 단정도 및 배정도 부동 소수점 수에 할당된 비트 수

단정도(단일정밀도): 32비트 배정도(2배정밀도): 64비트

float

double

03 실수 표현

정규화(normalization) : 과학적 표기방법

❖ 2진수의 정규화

$$\begin{aligned} 75.6875 &= 1001011.1011_{(2)} \\ &= 1.0010111011_{(2)} \times 2^6 \\ &= 1.0010111011_{(2)} \times 2^{110_{(2)}} \end{aligned}$$

1. 2진수로 변환한다
2. 정수부분이 1. 으로 되도록 소수점을 왼쪽으로 이동한다
3. 2의 지수부분도 2진수로 변환한다.
4. 부호(양수 0, 음수 1)
5. 지수(바이어스+지수부분)
6. 가수(1.을 생략한 숫자)

❖ 바이어스(bias) : 지수의 양수, 음수를 나타내기 위한 방법

- IEEE 754 표준에서는 바이어스 127(단정도) 또는 1023(배정도)을 사용
- 표현 지수 = 바이어스 + 2진 지수 값

부호	지수(바이어스 127)	가수(1.xxx ₍₂₎)
양수	01111111(127) + 110(6)	1.을 생략한 가수
0	10000101	001011101100000000000000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	---	---	---	---	---	---	---	---	---	---

32비트 단정도 (single precision) 부동소수점 표현.

03 실수 표현

❖ 10진수 -0.2를 단정도 부동소수점으로 표현

- 2진수로 변환하고 정규화한다.

$$\begin{aligned}-0.2 &= -0.00110011001100110011001..._{(2)} \\ &= -1.100110011001100110011001..._{(2)} \times 2^{-3} \\ &= -1.100110011001100110011001... \times 2^{-11(2)}\end{aligned}$$

부호	지수(바이어스 127)	가수($1.xxx_{(2)}$)
음수	$01111111(127) - 11(3)$	1.을 생략한 가수
1	01111100	10011001100110011001100

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	---	---	---	---	---	---	---	---	---	---

32비트 단정도 (single precision) 부동소수점 표현.

디지털 코드; 다른 이진 코드 체계

04 디지털 코드

1 BCD 코드(Binary Coded Decimal Code : 2진화 10진 코드, 8421 코드)

- BCD코드는 10진수 0(0000)부터 9(1001)까지를 2진화한 코드
- 표기는 2진수이지만 의미는 10진수
- 1010부터 1111까지 6개는 사용하지 않음

표 2-7 BCD(8421) 코드

10진수	BCD 코드	10진수	BCD 코드	10진수	BCD 코드
0	0000	10	0001 0000	20	0010 0000
1	0001	11	0001 0001	31	0011 0001
2	0010	12	0001 0010	42	0100 0010
3	0011	13	0001 0011	53	0101 0011
4	0100	14	0001 0100	64	0110 0100
5	0101	15	0001 0101	75	0111 0101
6	0110	16	0001 0110	86	1000 0110
7	0111	17	0001 0111	97	1001 0111
8	1000	18	0001 1000	196	0001 1001 0110
9	1001	19	0001 1001	237	0010 0011 0111

04 디지털 코드

- 예를 들어 10진수 237을 BCD 코드로 변환하면

10진수	2	3	7
	↓	↓	↓
BCD 코드	0010	0011	0111

- BCD 코드의 연산은 다음 예에서 ❶ ~ ❷와 같이 10진수처럼 연산한다.
- 계산 결과가 ❸과 같이 9를 초과하여 BCD 코드를 벗어나는 경우에는 결과에 6(0110)을 더한다.

❶ 10진수 덧셈	BCD 덧셈
6	0110 _(BCD)
+ 3	0011 _(BCD)
9	1001 _(BCD)

❷ 10진수 덧셈	BCD 덧셈
42	0100 0010 _(BCD)
+ 37	0011 0111 _(BCD)
79	0111 1001 _(BCD)

❸ 10진수 덧셈	BCD 덧셈
8	1000 _(BCD)
+ 7	+ 0111 _(BCD)
	1111 _(BCD)
	+ 0110 _(BCD)
15	0001 0101 _(BCD)

04 디지털 코드

3 그레이 코드

- 가중치가 없는 코드이기 때문에 연산에는 부적당하지만, 아날로그-디지털 변환기나 입출력 장치 코드로 주로 쓰인다.
- 연속되는 코드들 간에 하나의 비트만 변화하여 새로운 코드가 된다.

오류가 많은 환경,
통신, 보안 등
특수환경에서 사용

표 2-9 4비트 그레이 코드

10진 코드	2진 코드	그레이 코드	10진 코드	2진 코드	그레이 코드
0	0000	0000	8	1000	1100
1	0001	0001	9	1001	1101
2	0010	0011	10	1010	1111
3	0011	0010	11	1011	1110
4	0100	0110	12	1100	1010
5	0101	0111	13	1101	1011
6	0110	0101	14	1110	1001
7	0111	0100	15	1111	1000

3 → 4로 갈때
3개 비트가 바뀔

3 → 4로 갈때
1개 비트만 바뀔

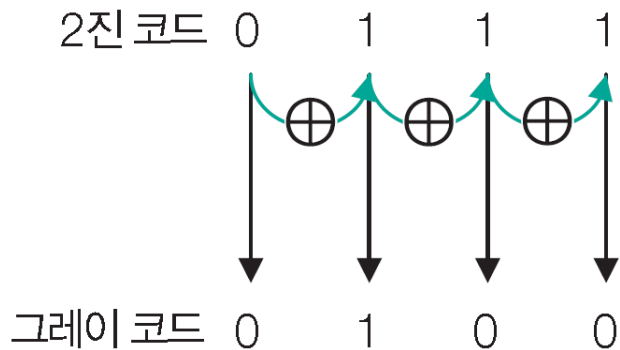
04 디지털 코드

❖ 2진 코드와 그레이 코드의 상호 변환 방법

$$F = A \oplus B$$

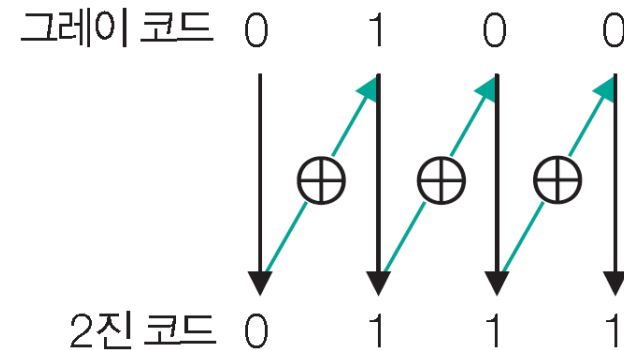
XOR의 진리표

A	B	F
0	0	0
0	1	1
1	0	1
1	1	0



(a) 2진 코드를 그레이 코드로 변환하는 방법

그림 2-7 2진 코드와 그레이 코드의 상호 변환 방법



(b) 그레이 코드를 2진 코드로 변환하는 방법

그레이코드 MSB를 그대로 2진코드로 내리고,
다음 비트와 비교(XOR)

04 디지털 코드

□ ASCII 코드

- 미국 국립 표준 연구소(ANSI)가 제정한 정보 교환용 미국 표준 코드
- 128가지의 문자를 표현 가능

표 2-13 ASCII 코드

(a) 코드 구성

b ₈	b ₇ b ₆ b ₅	b ₄ b ₃ b ₂ b ₁
패리티	존 비트	디지트 비트
1	3	4

(b) 표준 ASCII 코드표

*	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	TAB	LF	VT	FF	CR	SO	SI
1	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2		!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	₩	]	^	_
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{		}	~	DEL

□ 유니코드

- ASCII 코드의 한계성을 극복하기 위하여 개발된 인터넷 시대의 표준
- 다양한 운영체제와 최신의 모든 웹 브라우저 및 기타 많은 제품에서 유니코드를 지원한다.
- 미국, 유럽, 동아시아, 아프리카, 아시아 태평양 지역 등의 주요 언어들에 적용될 수 있다.
- 유니코드는 유럽, 중동, 아시아 등 거의 대부분의 문자를 포함하고 있으며, 10만개 이상의 문자로 구성되어 있다.
- 구두표시, 수학기호, 전문기호, 기하학적 모양, 덩벙 기호 등을 포함
- 앞으로도 계속해서 산업계의 요구나 새로운 문자들을 추가하여 나갈 것이다.

Korean (입력하기)	유니코드 (출력)
컴퓨터구조	\ucef4\ud4e8\ud130\uad6c\uc870

**에러 검출 코드; Error Correction Code,
부호화; Encoding,
복호화: Decoding**

05 에러 검출 코드

1 패리티 비트

- 짝수패리티(even parity) : 데이터에서 1의 개수를 짝수 개로 맞춤
- 홀수패리티(odd parity) : 1의 개수를 홀수 개로 맞춤
- 패리티 비트는 데이터 전송과정에서 에러 검사를 위한 추가비트
- 패리티는 단지 에러 검출만 가능하며, 여러 비트에 에러가 발생할 경우에는 검출이 안될 수도 있음

표 2-15 7비트 ASCII 코드에 패리티 비트를 추가한 코드

ASCII코드	짝수 패리티	홀수 패리티
⋮	⋮	⋮
A	0 1000001	1 1000001
B	0 1000010	1 1000010
C	1 1000011	0 1000011
D	0 1000100	1 1000100
⋮	⋮	⋮

05 에러 검출 코드

2 해밍 코드

- 에러를 정정할 수 있는 코드
- 추가적으로 많은 비트가 필요하므로 많은 양의 데이터 전달이 필요
- 데이터 비트와 **추가되는** 패리티 비트와의 관계
- 예를 들어 $d = 8$ 이면 $2^p \geq d + p + 1$ 을 만족하는 p 를 계산하면 4가 된다.
(즉, 8비트 데이터에 4비트 해밍 패리티 비트가 추가되어 12비트 코드가 됨)
- 해밍코드에서는 **짝수 패리티**를 사용

$$2^p \geq d + p + 1$$

P1은 한칸씩 띄워서 계산
P2는 두칸씩 띄워서 계산
P4는 네칸씩 띄워서 계산
P8는 여덟칸씩 띄워서 계산

표 2-16 해밍 코드에서 패리티 비트의 위치와 패리티 생성 영역

비트 위치	MSB											
	1	2	3	4	5	6	7	8	9	10	11	12
기호	P_1	P_2	D_3	P_4	D_5	D_6	D_7	P_8	D_9	D_{10}	D_{11}	D_{12}
P_1 영역	✓		✓		✓		✓		✓		✓	
P_2 영역		✓	✓			✓	✓			✓	✓	
P_4 영역				✓	✓	✓	✓					✓
P_8 영역								✓	✓	✓	✓	✓

05 에러 검출 코드

8비트 데이터의 에러 정정 코드

$$P_1 = D_3 \oplus D_5 \oplus D_7 \oplus D_9 \oplus D_{11}$$

$$P_2 = D_3 \oplus D_6 \oplus D_7 \oplus D_{10} \oplus D_{11}$$

$$P_4 = D_5 \oplus D_6 \oplus D_7 \oplus D_{12}$$

$$P_8 = D_9 \oplus D_{10} \oplus D_{11} \oplus D_{12}$$

$$P_1 = D_3 \oplus D_5 \oplus D_7 \oplus D_9 \oplus D_{11} = 0 \oplus 0 \oplus 0 \oplus 1 \oplus 1 = 0$$

$$P_2 = D_3 \oplus D_6 \oplus D_7 \oplus D_{10} \oplus D_{11} = 0 \oplus 1 \oplus 0 \oplus 1 \oplus 1 = 1$$

$$P_4 = D_5 \oplus D_6 \oplus D_7 \oplus D_{12} = 0 \oplus 1 \oplus 0 \oplus 0 = 1$$

$$P_8 = D_9 \oplus D_{10} \oplus D_{11} \oplus D_{12} = 1 \oplus 1 \oplus 1 \oplus 0 = 1$$

MSB											
P_1	P_2	D_3	P_4	D_5	D_6	D_7	P_8	D_9	D_{10}	D_{11}	D_{12}
		0		0	1	0		1	1	1	0

XOR의 진리표

A	B	F
0	0	0
0	1	1
1	0	1
1	1	0

표 2-17 원본 데이터가 00101110일 경우 해밍 코드 생성 예

비트 위치	1	2	3	4	5	6	7	8	9	10	11	12
기호	P_1	P_2	D_3	P_4	D_5	D_6	D_7	P_8	D_9	D_{10}	D_{11}	D_{12}
원본 데이터			0		0	1	0		1	1	1	0
생성된 코드	0	1	0	1	0	1	0	1	1	1	1	0

05 에러 검출 코드

□ 해밍 코드에서 패리티 비트 검사 과정

전송된 데이터 : 010111011110

											MSB
P_1	P_2	D_3	P_4	D_5	D_6	D_7	P_8	D_9	D_{10}	D_{11}	D_{12}
0	1	0	1	1	1	0	1	1	1	1	0

☞ 패리티들을 포함하여 검사

$$P_1 = P_1 \oplus D_3 \oplus D_5 \oplus D_7 \oplus D_9 \oplus D_{11} = 0 \oplus 0 \oplus 1 \oplus 0 \oplus 1 \oplus 1 = 1$$

$$P_2 = P_2 \oplus D_3 \oplus D_6 \oplus D_7 \oplus D_{10} \oplus D_{11} = 1 \oplus 0 \oplus 1 \oplus 0 \oplus 1 \oplus 1 = 0$$

$$P_4 = P_4 \oplus D_5 \oplus D_6 \oplus D_7 \oplus D_{12} = 1 \oplus 1 \oplus 1 \oplus 0 \oplus 0 = 1$$

$$P_8 = P_8 \oplus D_9 \oplus D_{10} \oplus D_{11} \oplus D_{12} = 1 \oplus 1 \oplus 1 \oplus 1 \oplus 0 = 0$$

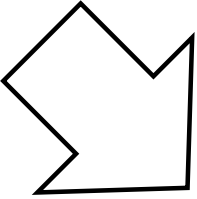
- 검사된 패리티를 $P_8 P_4 P_2 P_1$ 순서대로 정렬
- 모든 패리티가 0이면 에러 없음
- 하나라도 1이 있으면 에러 발생: 결과가 **0101**이므로 에러 있음
- 0101을 10진수로 바꾸면 **5**이며, 수신된 데이터에서 앞에서 5번째 비트 0101**1**1011110에 에러가 발생한 것이므로 0101**0**111110으로 바꾸어 주면 에러가 정정된다.

05 에러 검출 코드

표 2-17 원본 데이터가 00101110일 경우 해밍 코드 생성 예

비트 위치	1	2	3	4	5	6	7	8	9	10	11	12
기호	P_1	P_2	D_3	P_4	D_5	D_6	D_7	P_8	D_9	D_{10}	D_{11}	D_{12}
원본 데이터			0		0	1	0		1	1	1	0
생성된 코드	0	1	0	1	0	1	0	1	1	1	1	0

송신



수신

표 2-18 해밍 코드에서 에러가 발생한 경우 정정

비트 위치	1	2	3	4	5	6	7	8	9	10	11	12
기호	P_1	P_2	D_3	P_4	D_5	D_6	D_7	P_8	D_9	D_{10}	D_{11}	D_{12}
해밍 코드	0	1	0	1	1	1	0	1	1	1	1	0
패리티 검사	1	0		1				0				
$P_8P_4P_2P_1 = 0101_{(2)} = 5_{(10)}$: 5번째 비트에 에러가 발생. 1 → 0으로 정정												
해밍 코드 수정	0	1	0	1	0	1	0	1	1	1	1	0

05 에러 검출 코드

3 순환 중복 검사(CRC)

- 높은 신뢰도를 확보하며 에러 검출을 위한 오버헤드가 적고, 랜덤 에러나 버스트 에러를 포함한 에러 검출에 매우 좋은 성능을 갖는다.

❖ CRC 발생기 및 검출기

k가 16이면 CRC-16

- 송신 측에서는 'd비트 데이터를 k비트만큼 올리고' '(k+1)비트로 나눠' 'k비트의 나머지를 합쳐' **d + k 비트를 전송**
- 수신 측에서는 수신된 **d + k 비트**의 데이터를 **키 값**으로 나누었을 때 **나머지가 0이면 에러가 없는** 것이지만, 0이 아니면 에러가 발생한 것으로 판단한다.

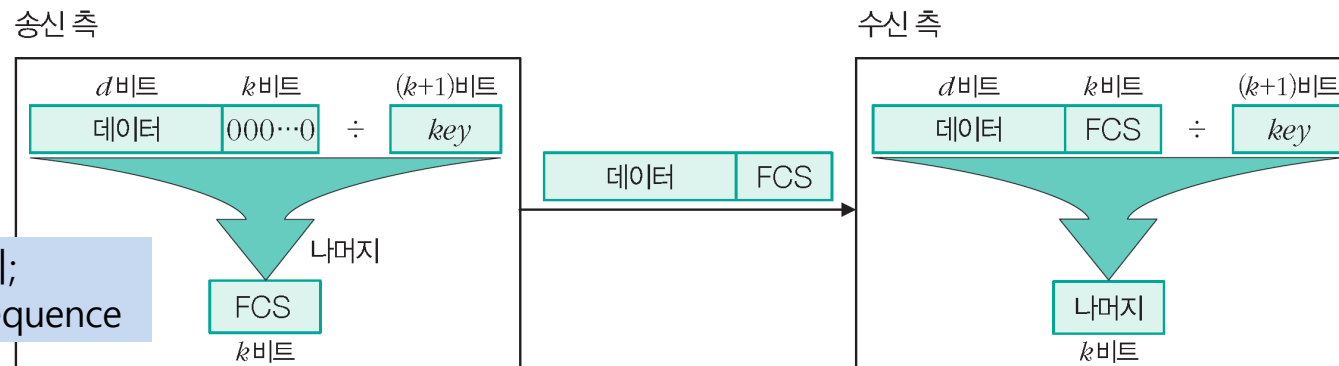


그림 2-8 CRC 발생기와 검사기

05 에러 검출 코드

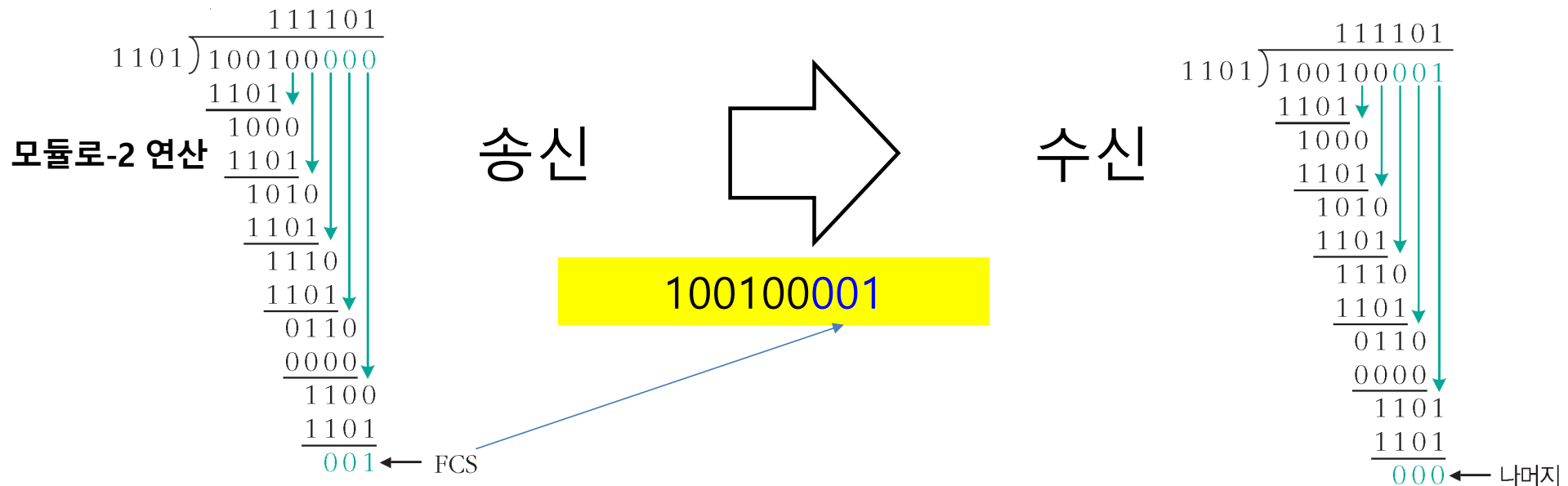
❖ CRC 계산에 사용되는 모듈로-2 연산

- 사칙 연산에서 캐리는 고려하지 않는다.
- 덧셈 연산은 뺄셈 연산과 결과가 같으며 XOR 연산과도 같다.

모듈로-2 연산

- 덧셈 연산: $0+0=0$, $0+1=1$, $1+0=1$, $1+1=0$
- 뺄셈 연산: $0-0=0$, $0-1=1$, $1-0=1$, $1-1=0$

- 데이터가 100100이고, 키 값이 1101인($k+1 = 4$ 비트) 경우 FCS를 계산하는 예



(a) CRC 발생기에서 계산 과정

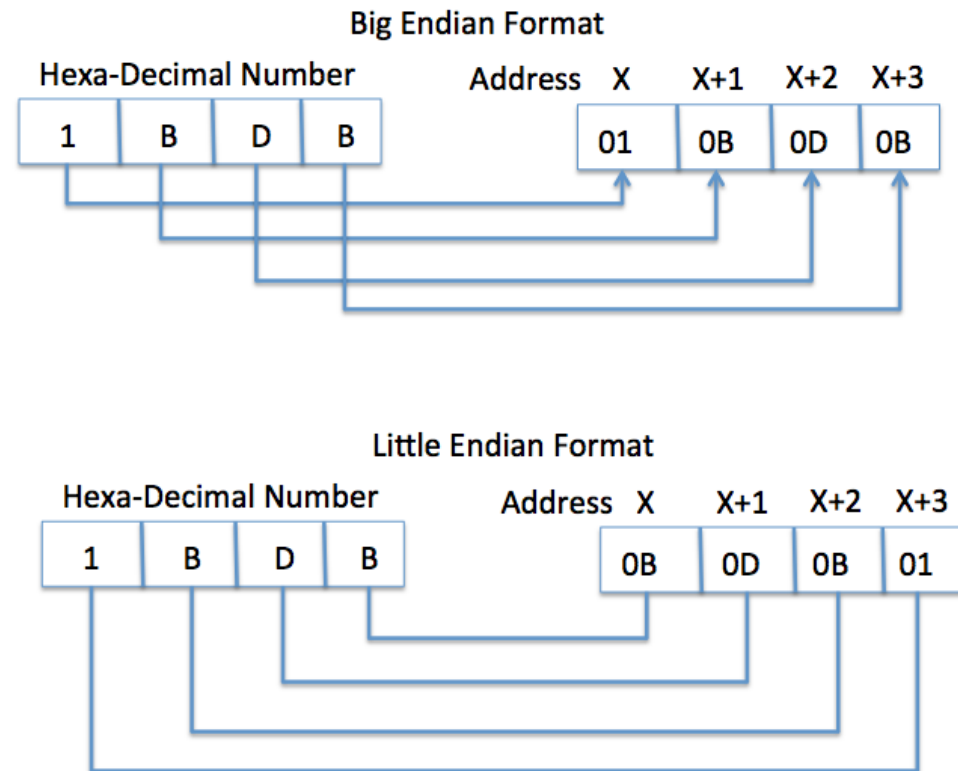
(b) 검사기에서 계산 과정

그림 2-9 CRC 발생기와 검사기에서 2진 나눗셈

Byte Ordering

- Byte Ordering: Byte 단위로 메모리 저장 또는 통신 전송 되는 순서
 - 2 바이트 이상인 경우부터 차이가 남
- Big-Endian
 - 상위 바이트가 하위 번지에 저장
 - RISC 계열의 CPU
 - 네트워크 통신(TCP/IP 등)
- Little-Endian
 - 하위 바이트가 하위 번지에 저장
 - Intel 계열의 CPU

Big Endian and Little Endian Formats



감사합니다.